

What we did

RL on #230's <contacts-v1.multi> checkpoint, with the reward computed over the **sections of a single rollout** rather than over the rollouts of a group.

Nine reward designs, thirteen runs, **55 scored checkpoints**, all on 8 × A100 via SkyRL: **M-C** each section's contribution to its own rollout's consensus · **M-F** the last section's F1 · **M-B** the best section's F1 (oracle) · **M-BC** best + consensus · **M-FC** last + consensus · **M-K** the rollout's own consensus R-precision · **M-BP** M-B plus a candidate-count floor · **M-KS/2/3** M-K plus a within-rollout shaping term · **M-O** a zero-LR control.

Why

#208 ran eleven runs across five rewards and none improved consensus R-precision. The mechanism was a **unit mismatch**: the reward scored one rollout, while the metric votes over 100 *independent* rollouts — an object no rollout can see.

Under <contacts-v1.multi> one rollout emits ~22 contact sets in a single generation, so a reward on those is computed on the kind of object the metric scores, with credit assignment inside the sequence.

The result

Best consensus 0.5806 (M-K), best oracle 0.5663 (M-B), best final section 0.5267 (M-C) — three criteria, three different arms. M-K is the only arm to improve **all four** aggregation modes with every CI excluding zero, on every cut.

The primary criterion is not met. 22 independent plain rollouts read 0.5896; M-K is 0.0090 short. At a larger budget the two draw level — M-K pooled 0.6098 against plain-100's 0.6058, a paired CI that includes zero. So RL **closed the gap between the multi format and ordinary sampling; it did not clearly open one.**

#208's negative result is a dose-response, not a verdict

Consensus against distance moved: 0.5673 at KL 0, **0.5806 at 0.016**, 0.5529 at 0.031, 0.3969 at 0.486. Every reward helps at small KL and damages at large.

#208 ran its arms at KL 0.06–0.10 and to 3.96 — past the peak on all of them — and its two arms under 0.0015 never moved. **The window it needed lay between the two learning rates it tried.** Two runs at a 3.3× different rate agree to 0.002 at matched KL, so this is distance, not schedule.

Reward design, in three parts

$E[r] = 0$ is necessary and not sufficient. M-C's per-section marginal is **366× larger at 1 section than at 22** — it paid the policy to emit a worse answer — while $E[A] = 0$ held exactly, because centring is computed *inside* the quantity being gamed.

Every arm moved its candidate count in the direction its own reward pointed. M-C's gradient is strongly negative (collapsed to 1.1 sections); M-B's strongly positive and self-paying (grew to ~26 and held); M-F's nearly flat — and **a weak gradient is not a safe one, it is an unconstrained one**: M-F ran to 259 sections carrying 1.4 contacts each.

Gates written against the failure you have seen will miss the next one. All three original criteria are one-sided; M-F's failure pushed every one *away* from its threshold and none fired.

The arm that won was designed from a failure

M-C's collapse was traced to a scale bug: its per-section marginal is 366× larger at one section than at 22, so it paid the policy to emit fewer candidates. The diagnosis named the repair — a **rollout-level** base that *falls* when sections are dropped.

M-K is that base with nothing else on top: reward = the rollout's own consensus R-precision, GRPO-centred. Section count then held flat at ~22 for 48 steps, and it produced the experiment's best consensus. It is also the only arm neither killed by a gate nor diverged — it ran out of scheduled steps.

Within-sequence credit assignment was the hypothesis's mechanism, and it turned out not to be needed. What was needed was for the reward to be computable on the object the metric scores — which is what the multi format makes possible.

The one lever that moved a number

Every arm rewards candidates for being individually good. None pays for them being *different from each other* — and the metric is a vote that feeds on coverage: one multi rollout covers 658 distinct pairs against 1,065 for 22 independent ones.

M-KS2 adds a zero-sum, positionally-corrected term crediting each section for what it added given its predecessors. Result: ORACLE-best **0.5602 to 0.5677** (+0.0074, CI excluding zero), consensus unchanged (-0.0007, a tie), and quality shifted EARLIER in the rollout (second_last +0.0121, last -0.0131).

The first version of it died fastest of any arm here — a "zero-sum" guarantee that bounded the rollout's LEVEL while leaving its SHAPE a decaying function of section index, which is a stop-early signal because a section owns the token that opens it. The measurement found it; the positional correction fixed it.

Two results that outlive the experiment

Selection is dominated by aggregation. An ORACLE selector of one draft reads 0.5646 where simply voting the same drafts reads 0.5750. A final section should synthesise its predecessors, not pick among them.

The spread is the resource. Making candidates more uniform raises mean section F1 from 0.432 to 0.532 and *lowers* best-of-22 — the same trade that defeats every sharpening reward here, appearing in the corpus's own size law.

Long trajectories do not beat short ones

Three unrelated interventions, all aimed at training past the peak:

A smaller learning rate. M-B at lr 3e-6 resumed to step 180 — killed, section count collapsed to 11.0. Its flat evals at steps 60-120 were a plateau *in steps* while KL climbed 0.0087 to 0.0397. More steps is more distance.

A KL penalty. kl_loss_coef 0.001 to 0.05. At *matched distance*, 3.3x the optimisation steps scored **0.0094 worse** — and the penalty turned out to buy about what a 3x learning-rate cut buys rather than holding distance at all.

A candidate-count floor on M-B's reward. It worked: the collapse vanished and it beat the unpenalised run by **+0.0178** at step 150. Then it died three steps later of *diversity* collapse instead — the reward can be raised by emitting fewer good sections or many copies of one, and closing the first route opened the second.

Each slowed or reshaped the degradation. **None produced a checkpoint better than the peak it started from.**

What next

Nothing further on "train it longer." Three interventions, three ways of failing to beat the peak. Whatever bounds this is not the step size, not the distance per step, and not the candidate count.

Explain the peak before designing another reward. Every arm peaks at KL 0.009-0.016 and every intervention only changes the approach to it. That property is the finding worth chasing.

The shaping and blend terms remain untested. M-K is the $\beta = \lambda = 0$ corner of the derived arm, and the base alone beat every shaped and blended arm.

The diversity gap is a corpus question. One rollout's sections cover 658 distinct pairs against 1,065 for 22 independent rollouts.

Figures

curves_accuracy.png — all 37 scored checkpoints against training step.

curves_reward.png — each arm against its own reward.

dose_response.png — R-precision against distance moved.

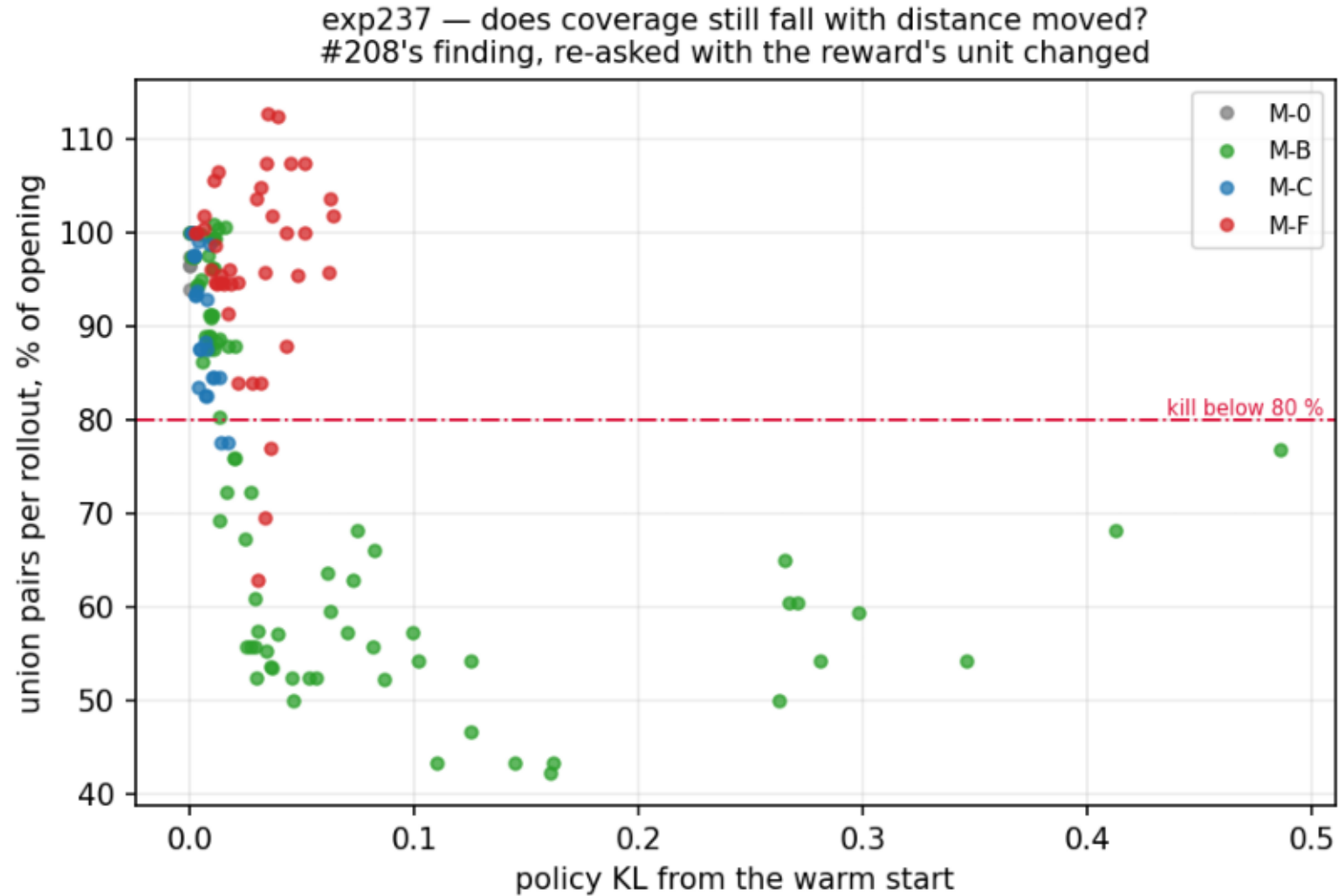
reward_vs_section_count.png — each reward's gradient in the count direction.

section_count_incentive.png — M-C's scale pathology, 366× at one section.

gates_over_training.png · reward_shape.png · coverage_vs_kl.png

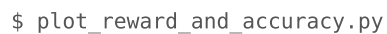
coverage_vs_kl.png

Union coverage against distance moved. M-C crosses the 80% floor by KL 0.013, M-B by 0.016, M-F not until 0.036 -- same destination, different cost per unit of KL.



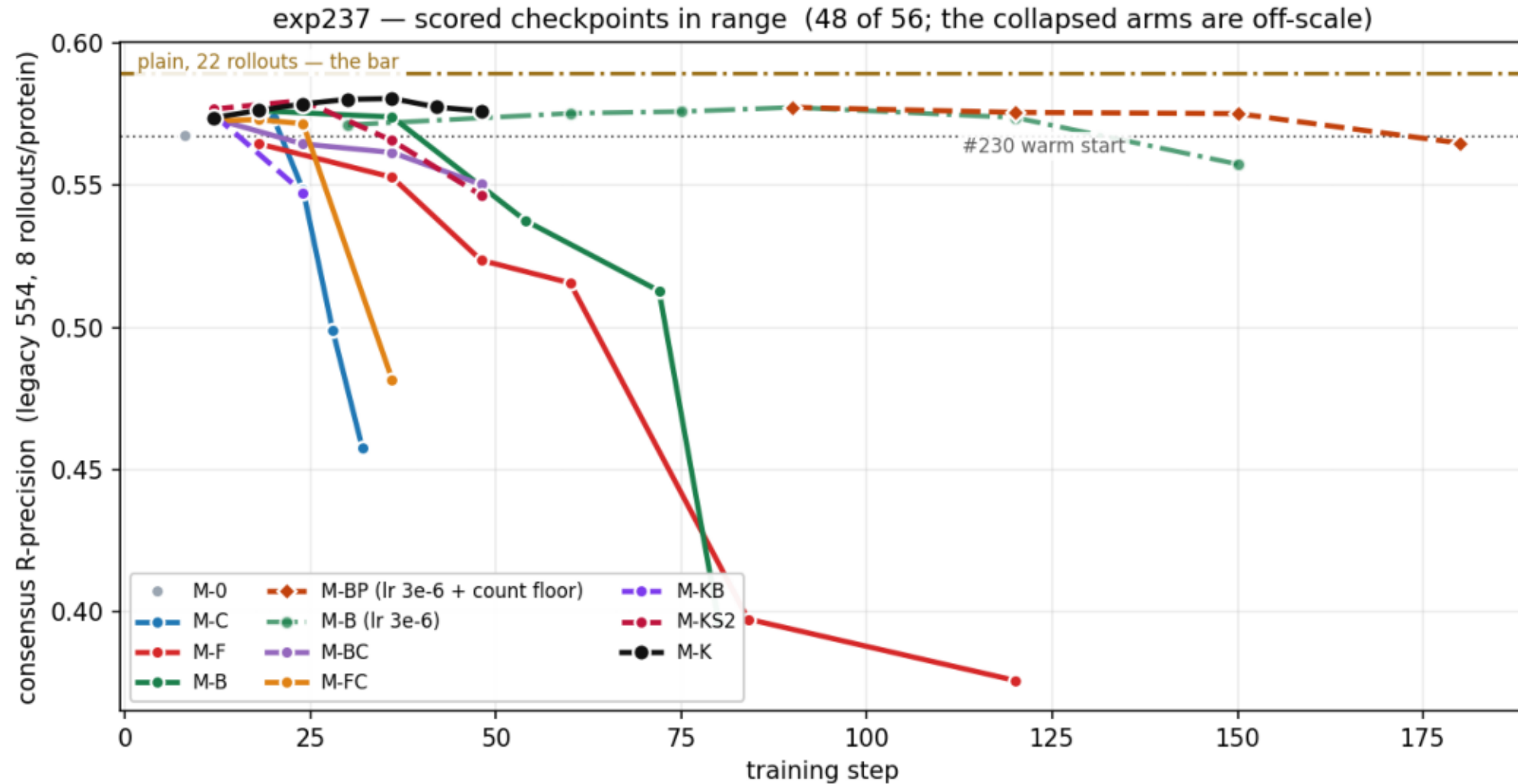
```
$ make_plots.py --steps data/training_steps.csv.gz --out plots/
```

Consensus R-precision on the 554-protein exp89 benchmark at all 37 scored checkpoints. Five arms peak by step ~20 and fall away; M-K peaks later, at step 30-36, and highest.



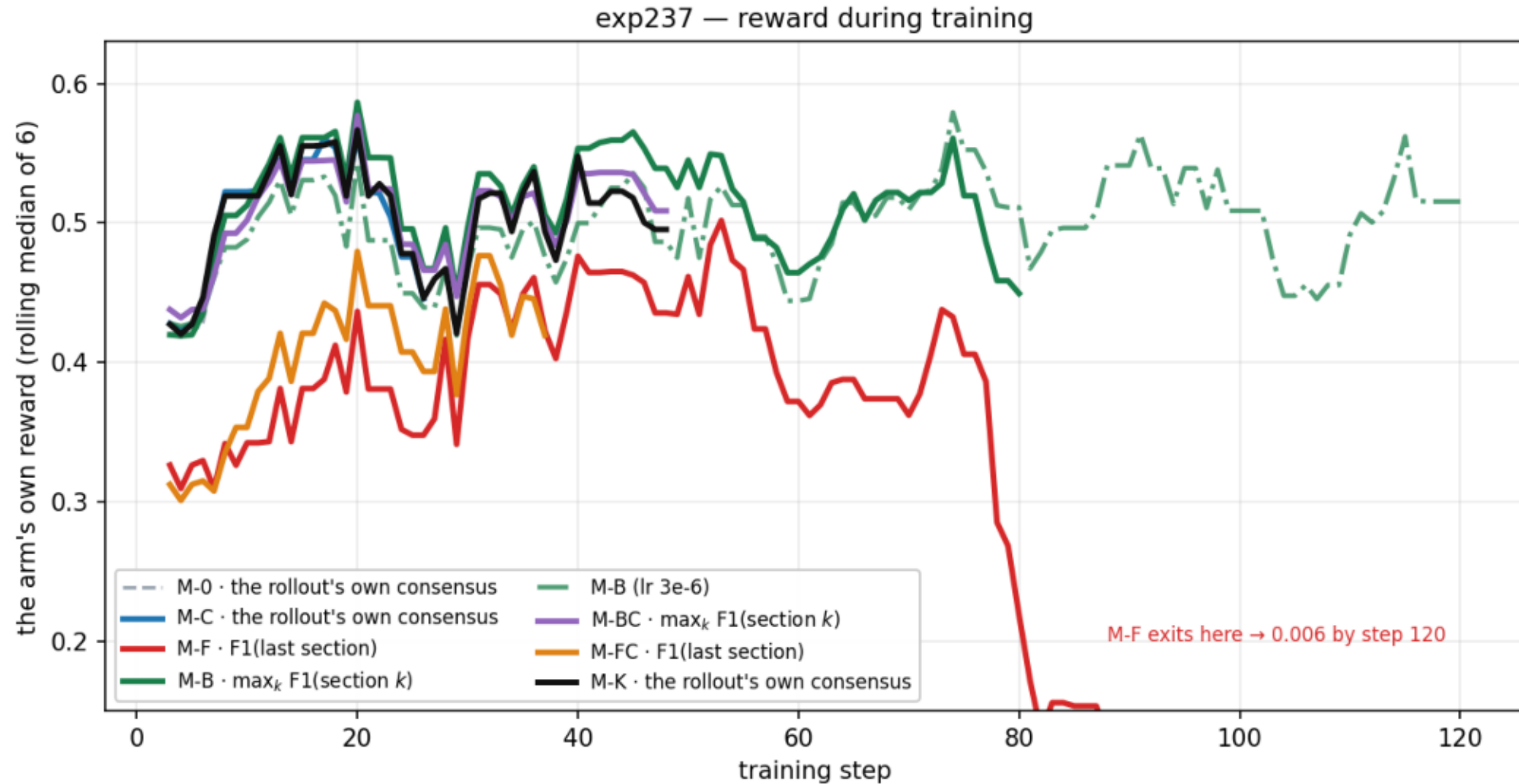
curves_accuracy_bare.png

Consensus R-precision on the 554-protein exp89 benchmark at all 37 scored checkpoints. Five arms peak by step ~20 and fall away; M-K peaks later, at step 30-36, and highest.



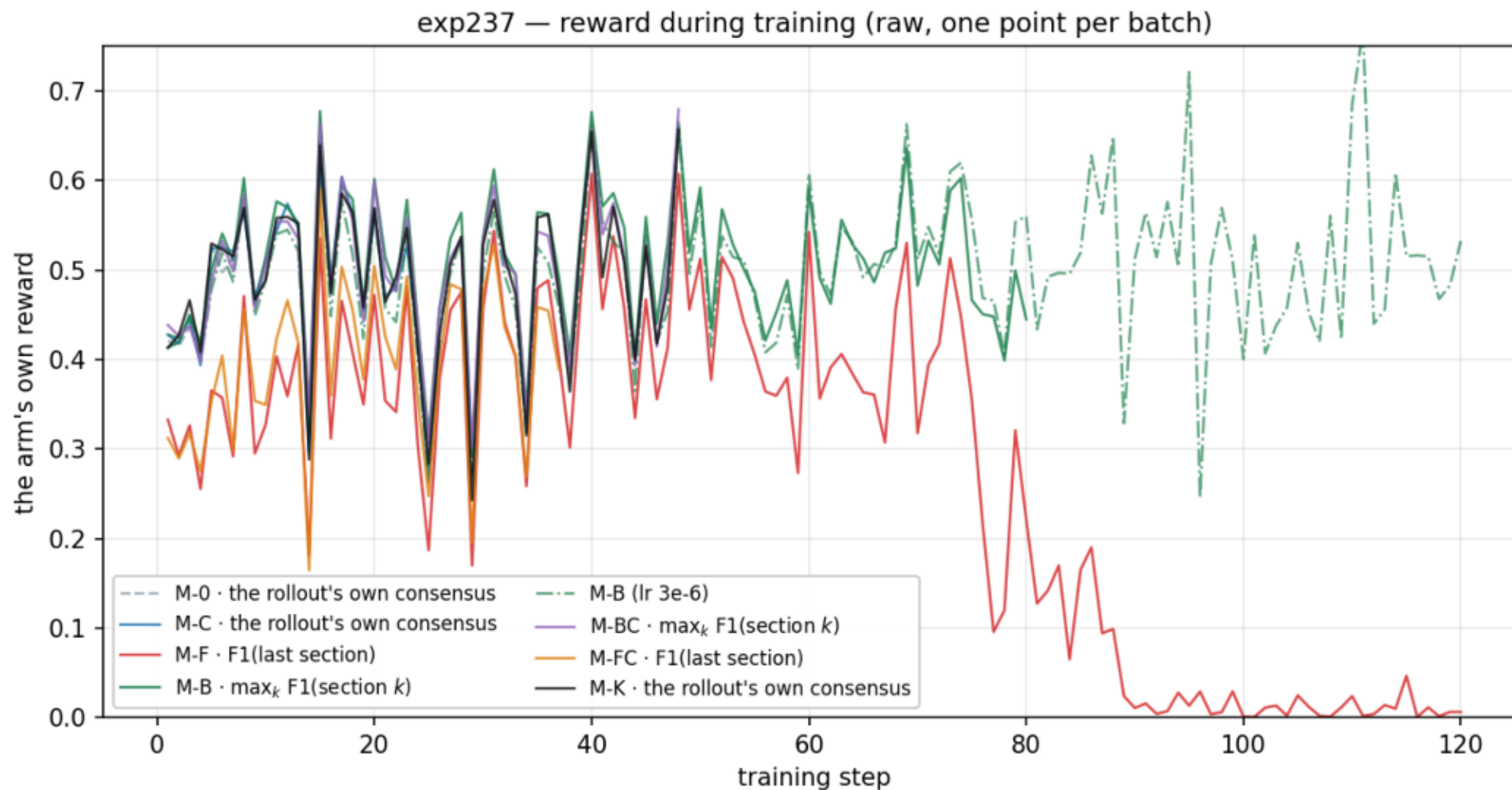
curves_reward.png

Each arm against its OWN reward — the arms do not share one, so the y-axis is not a common scale. Rolling median of 6 training batches; the raw per-batch series is dominated by the protein draw.



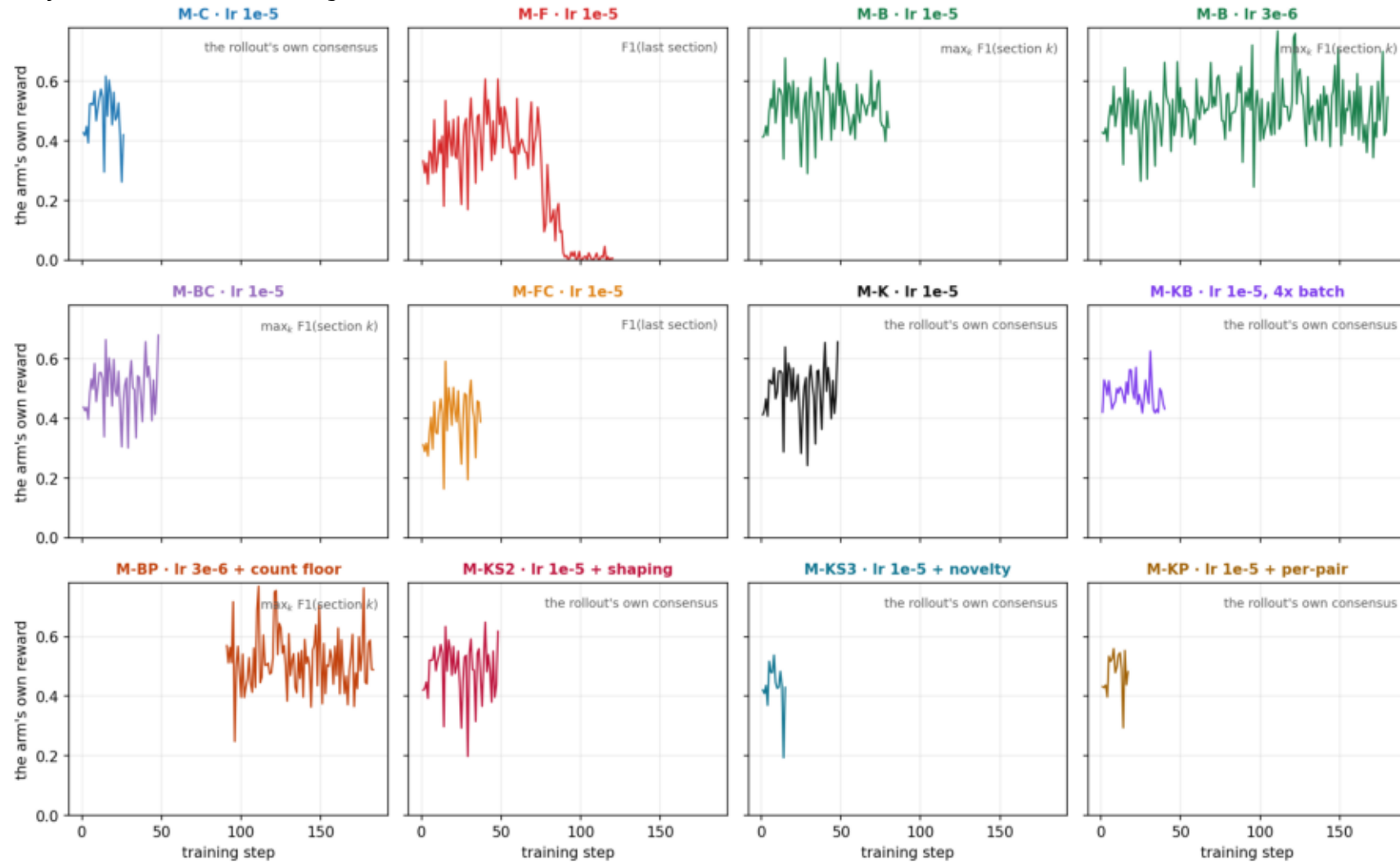
curves_reward_bare.png

Each arm against its OWN reward — the arms do not share one, so the y-axis is not a common scale. Raw values, one point per training batch (8 proteins x 8 rollouts); the batch-to-batch spread is dominated by the protein draw.



curves_reward_facets.png

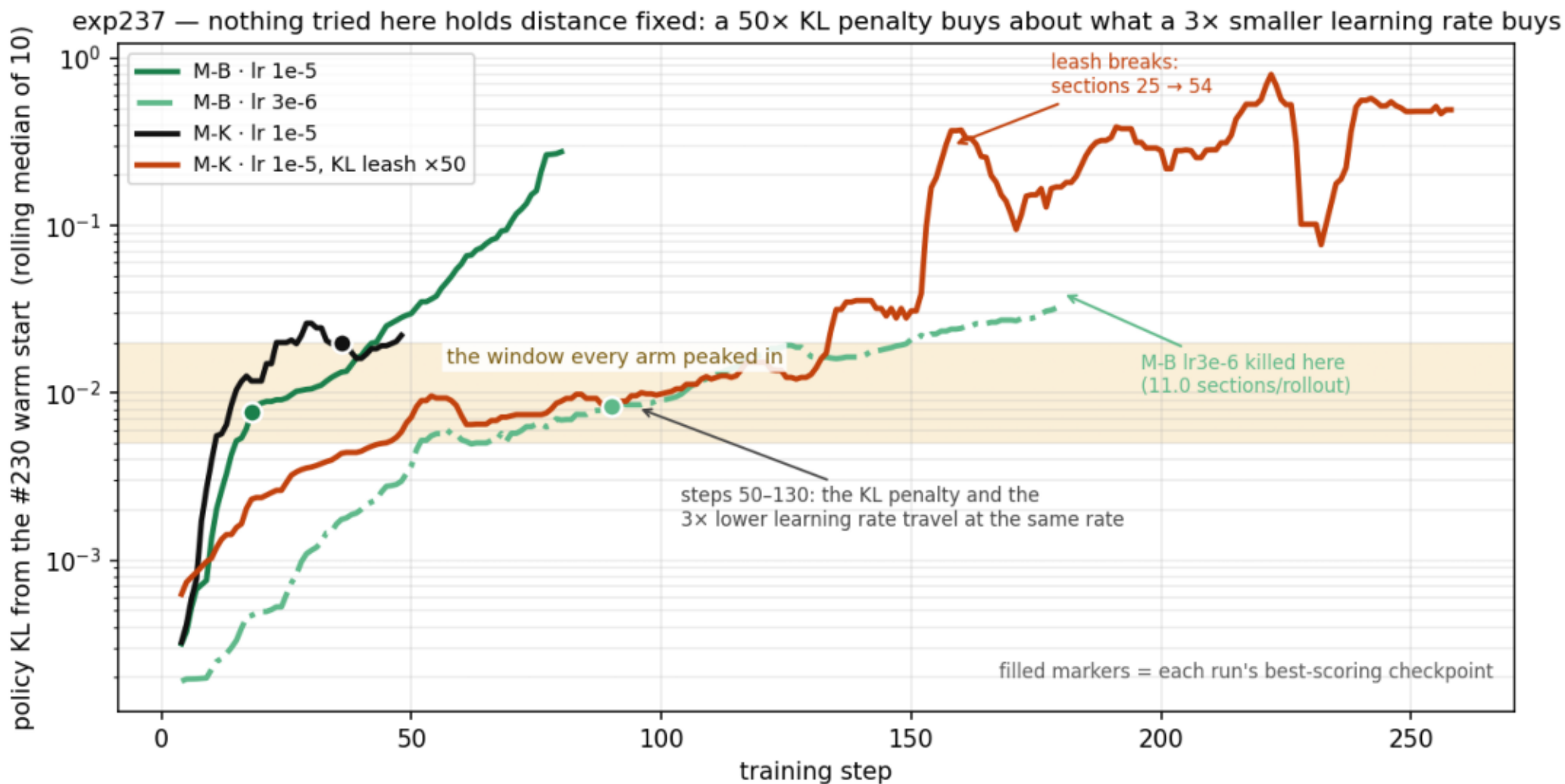
Raw per-batch reward, one panel per run, no smoothing. Each arm is plotted against its OWN reward — the quantity is named in each panel — so the shared y-axis is a common range, not a common scale.



\$ plot_reward_facets.py

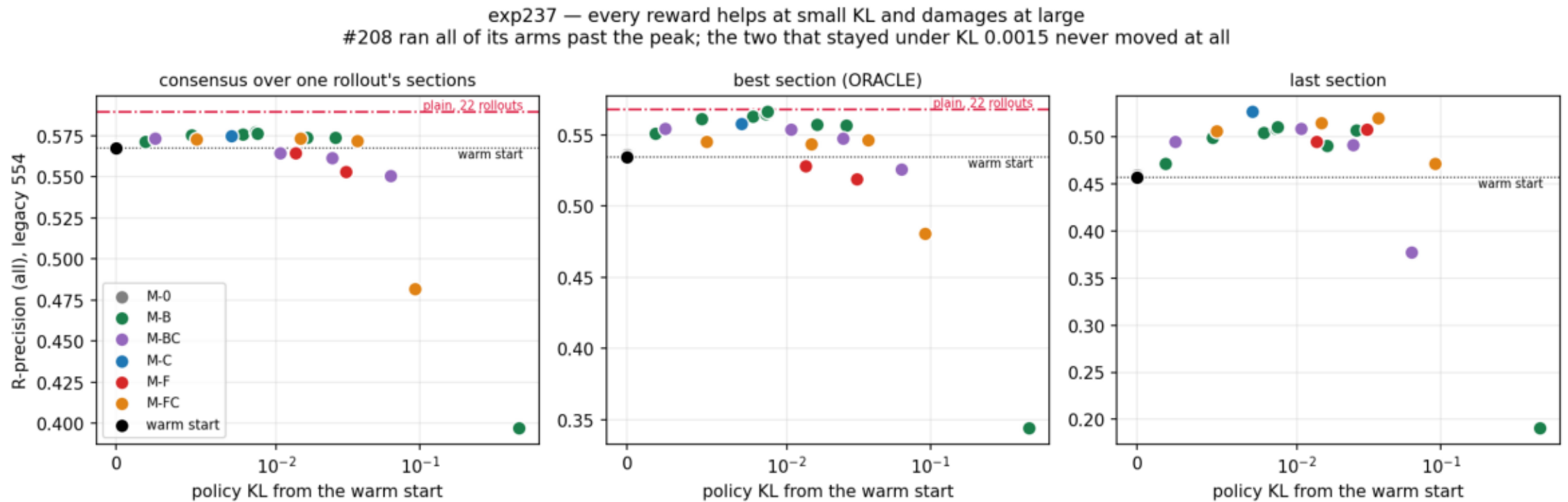
distance_vs_steps.png

Distance moved against steps taken. A 3.3x smaller learning rate takes ~5x the steps through the same KL window and then fails the same way. Raising kl_loss_coef 50x does NOT hold distance fixed — from step 50 to 130 it tracks the lr-3e-6 run, then runs away to KL 0.5.



dose_response.png

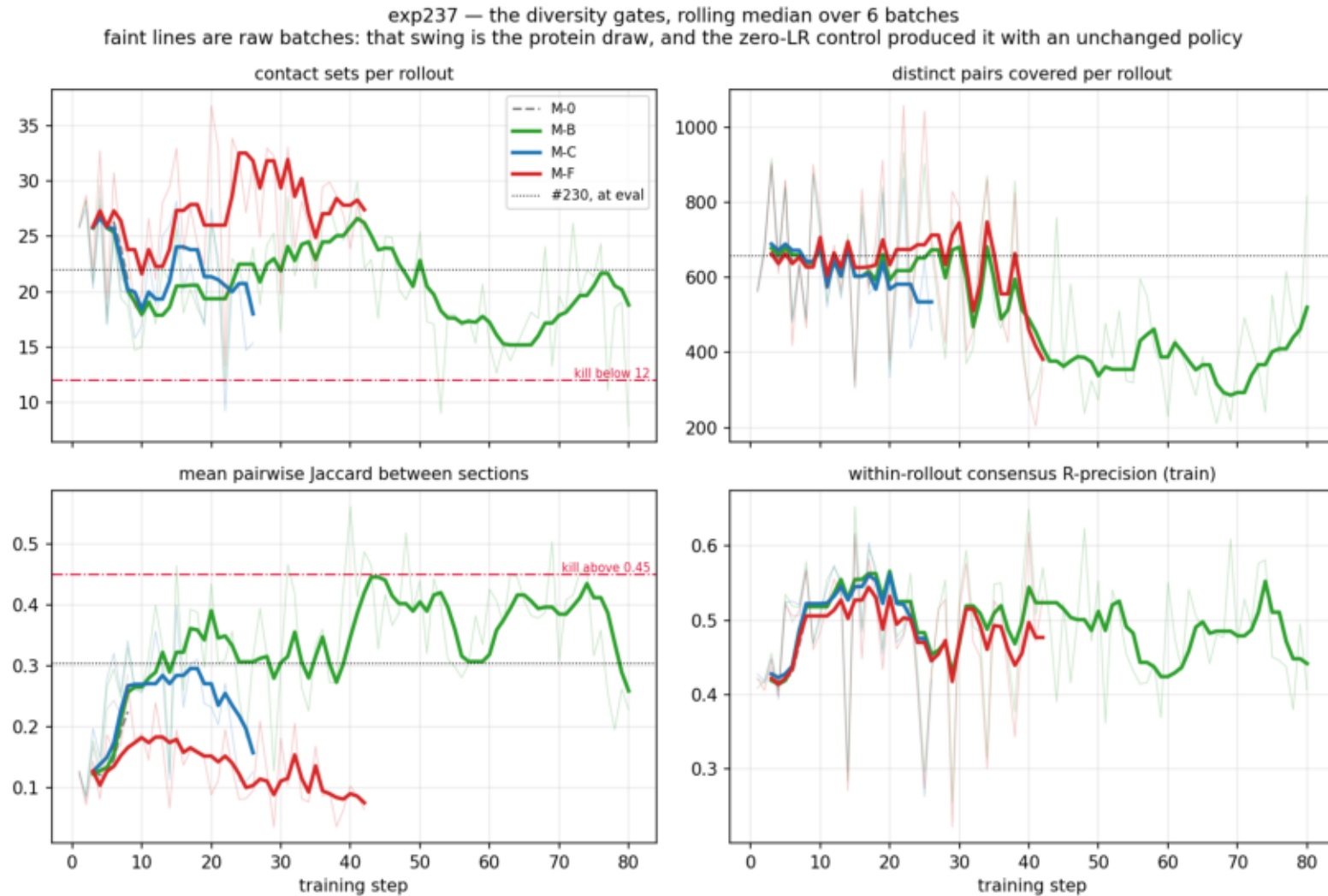
R-precision against distance moved. Consensus peaks at KL 0.007-0.016; #208 ran every arm past it. Red = the budget-matched bar, which nothing reaches.



```
$ plot_reward_shape.py --data data --out plots
```

gates_over_training.png

The diversity gates per batch, rolling median over 6. Jaccard separates the arms: M-B rises, M-C and M-F fall. Faint = raw batches, i.e. the protein draw.

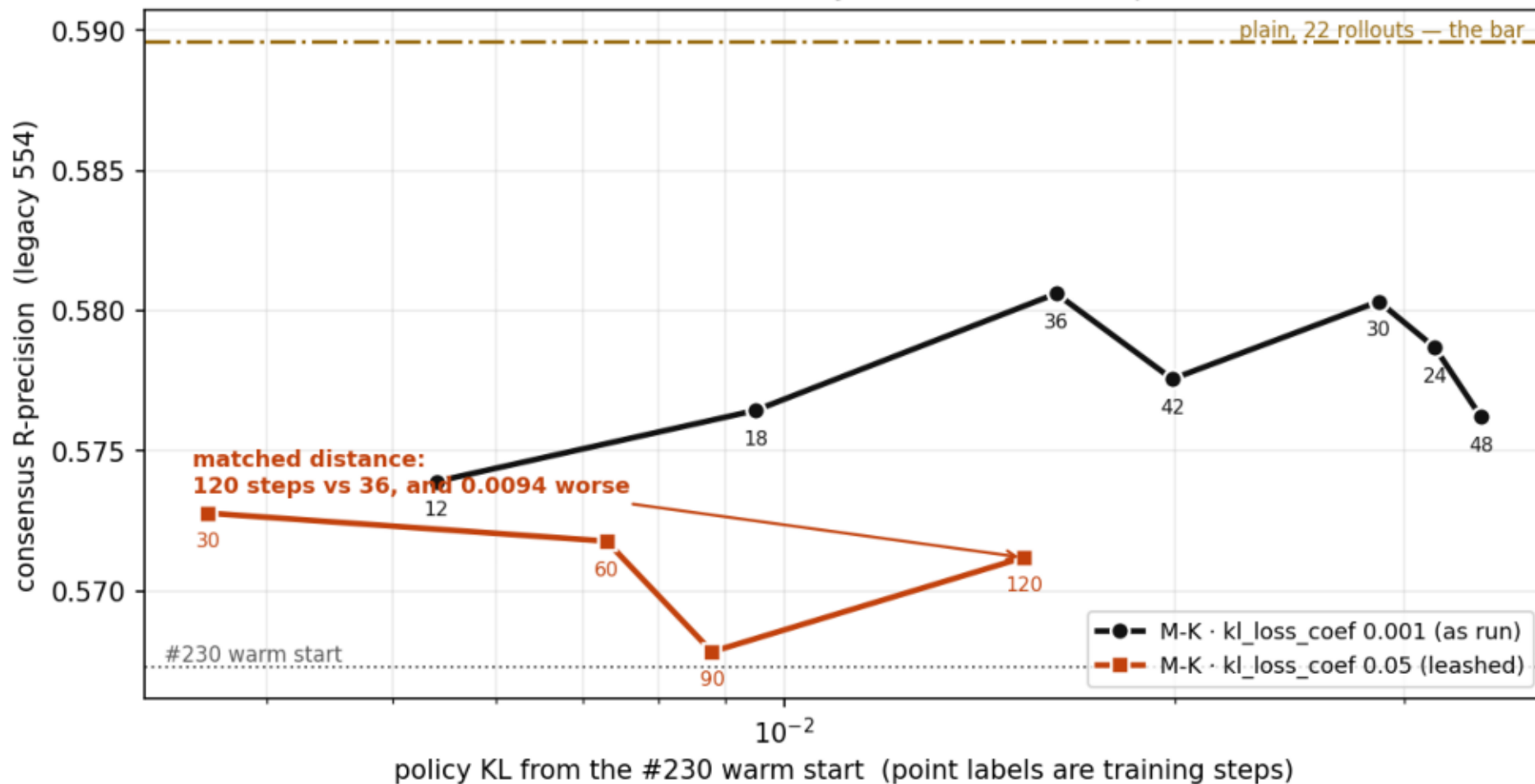


```
$ make_plots.py --steps data/training_steps.csv.gz --out plots/
```

leash_vs_free.png

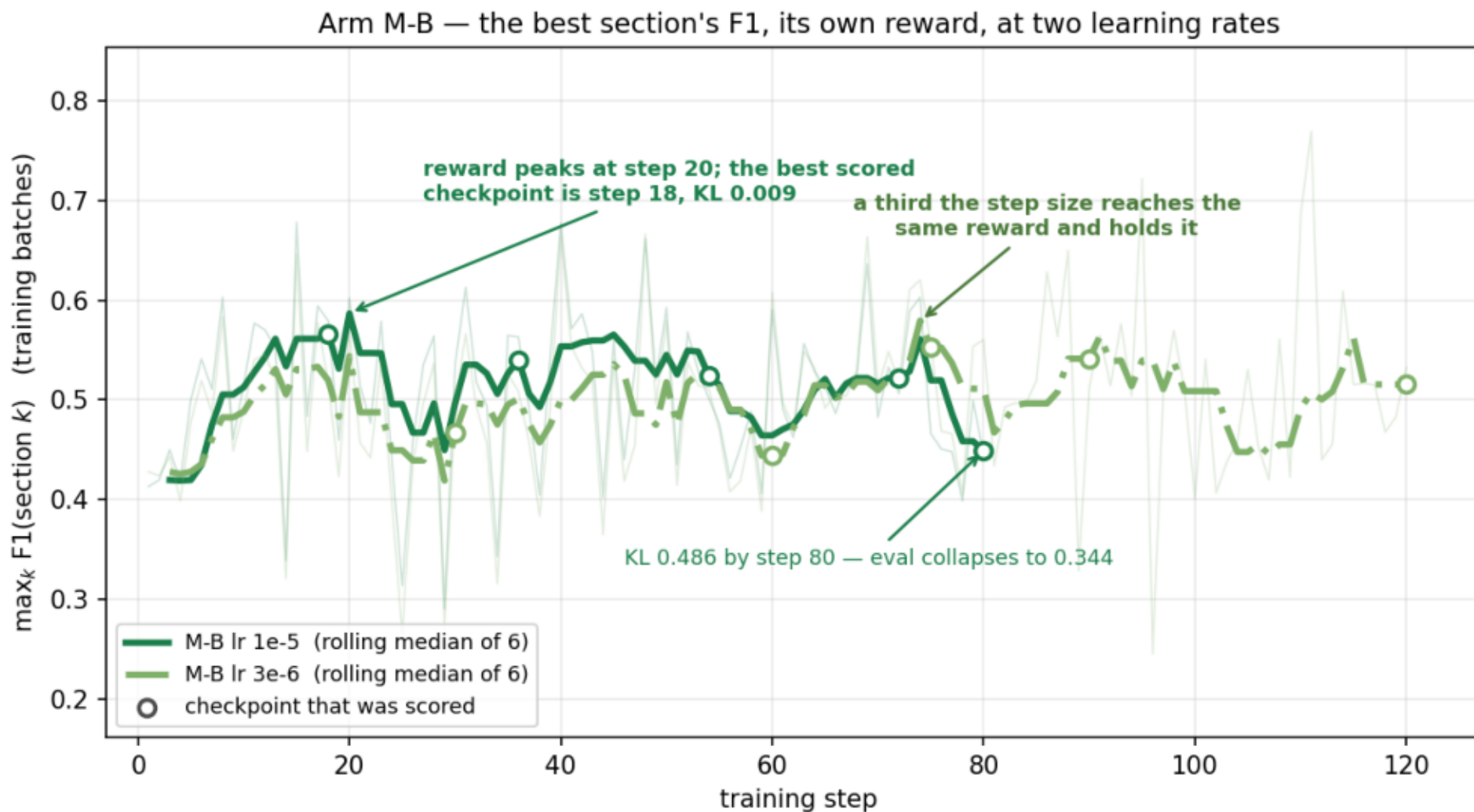
Arm M-K with and without a 50x KL penalty, plotted against distance moved so the extra steps are a vertical comparison at fixed x. The leashed trace is below the unleashed one at every matched distance and is flat across steps 30-120 — the path to a distance is not interchangeable.

exp237 — same reward, same lr, 3x the steps to the same distance:
the leashed run is uniformly worse and never improves



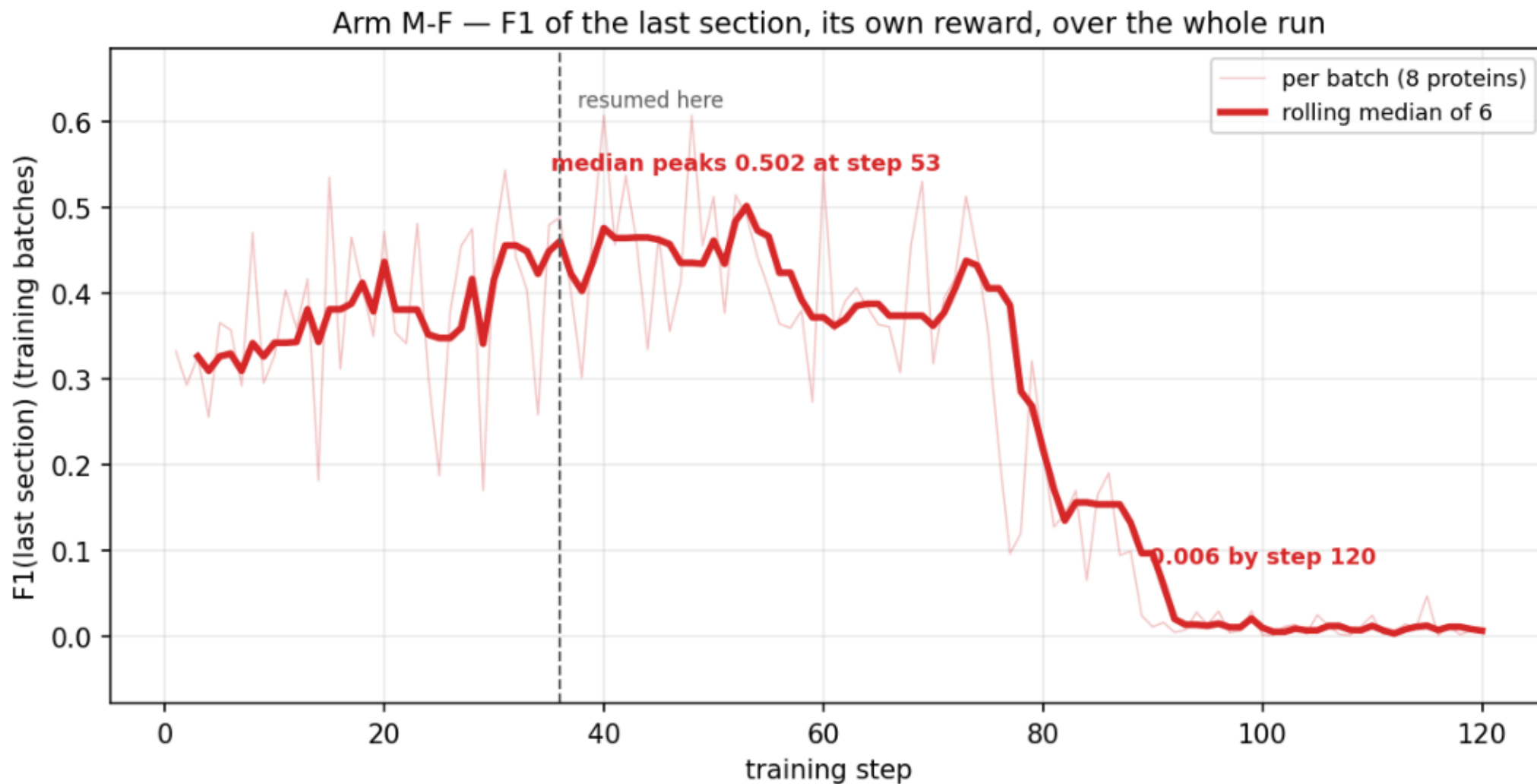
reward_curve_m_b.png

M-B's own reward at lr 1e-5 and 3e-6. Both peak near 0.58; the slow walk holds it while the fast one runs to KL 0.486 and its eval collapses to 0.344. Open circles mark the checkpoints that were scored.



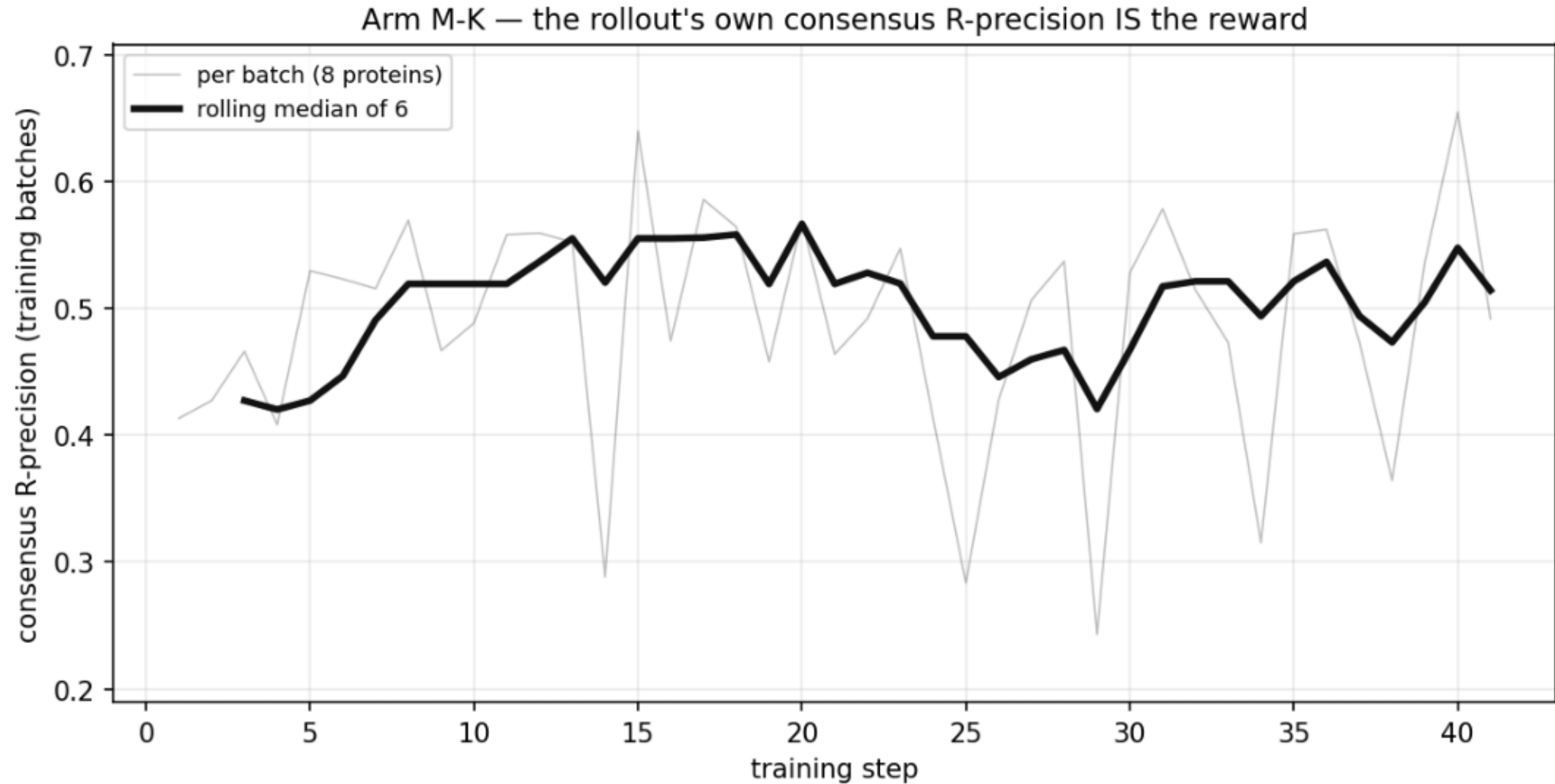
reward_curve_m_f.png

M-F's reward over both runs. It peaks at step 48 and is at 0.006 by step 120: the policy found the unconstrained direction (259 sections of 1.4 contacts each) and took the reward down with it.



reward_curve_m_k.png

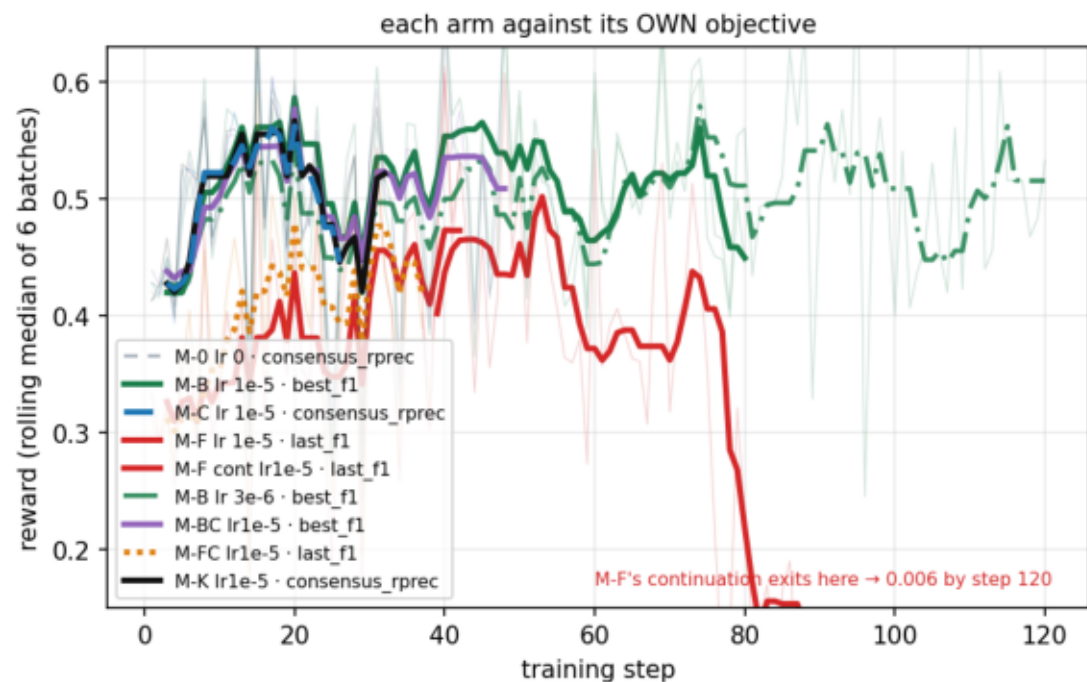
M-K's reward is exactly the quantity plotted: the rollout's own consensus R-precision, one scalar per rollout with a GRPO baseline.



reward_curves.png

Each arm against its own reward (left) and the shared consensus axis (right), rolling median of 6 batches. Training-batch values on 8 proteins, not eval.

exp237 — reward over training. Faint lines are raw batches: that swing is the protein draw, and the lr-0 control produced it with a policy that never changed.

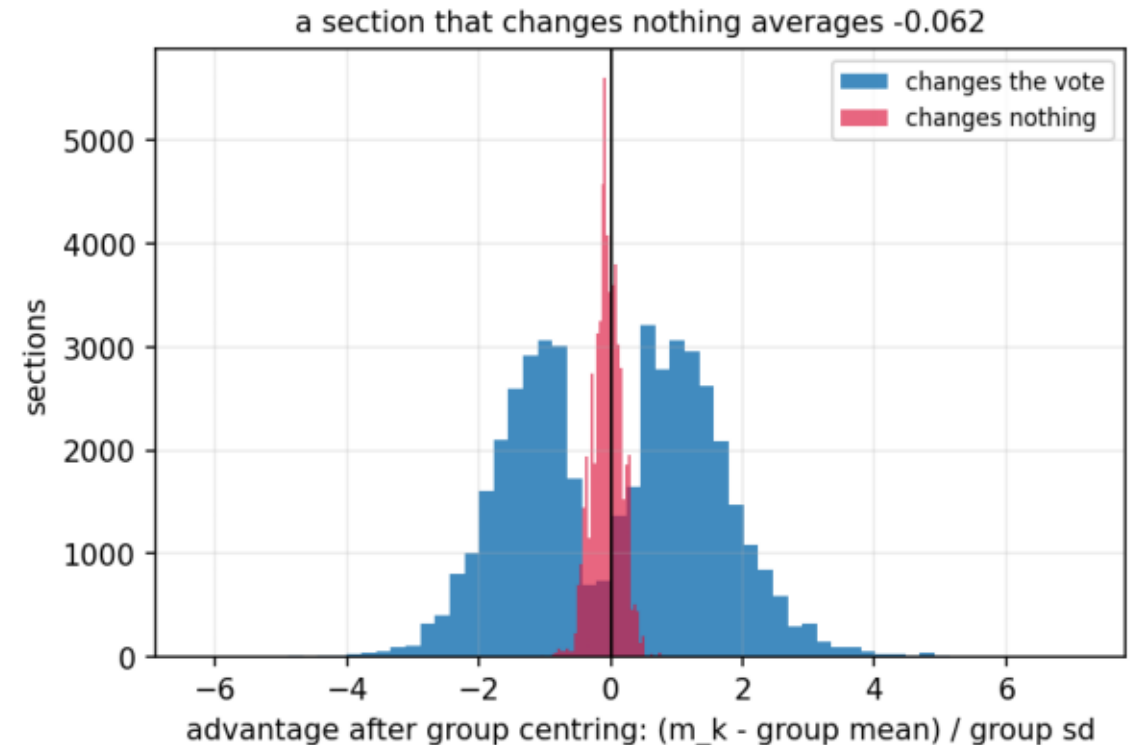
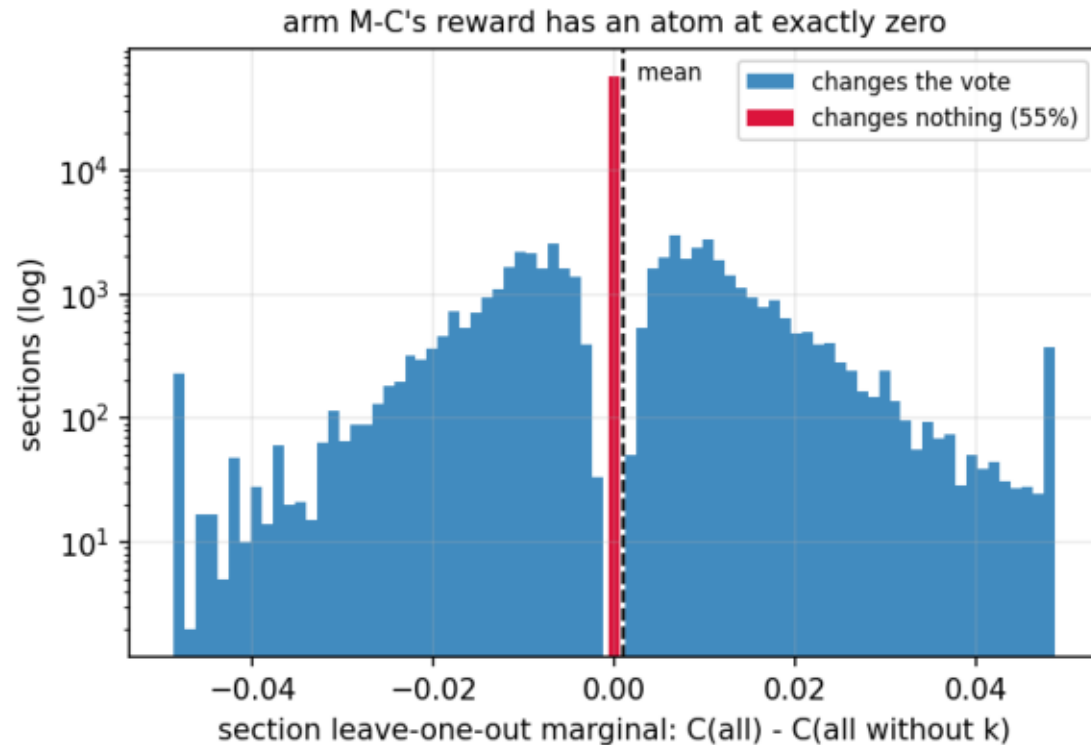


```
$ plot_reward_curves.py --csv data/training_steps.csv.gz --run M-F cont lr1e-5=/tmp/claude-1000/logs_final/exp237_m_f_lr1e-5.log --offset M-F cont lr1e-5=36 --run M-B lr3e-6=/tmp/claude-1000
```

reward_shape.png

Arm M-C's reward before training: 55% of sections change no vote, and average -0.062 once centred. M-F has no such atom and collapsed the same -- so this is not the cause. See RESULTS.md.

exp237 — why $E[A] = 0$ holding exactly was not enough

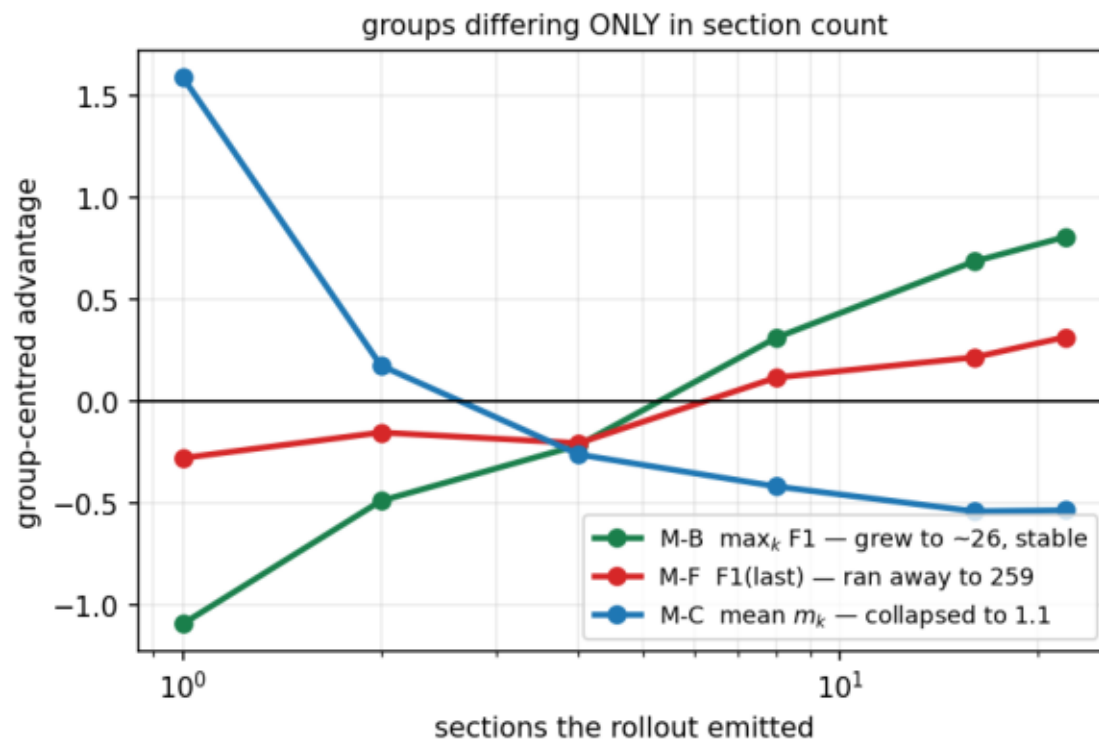
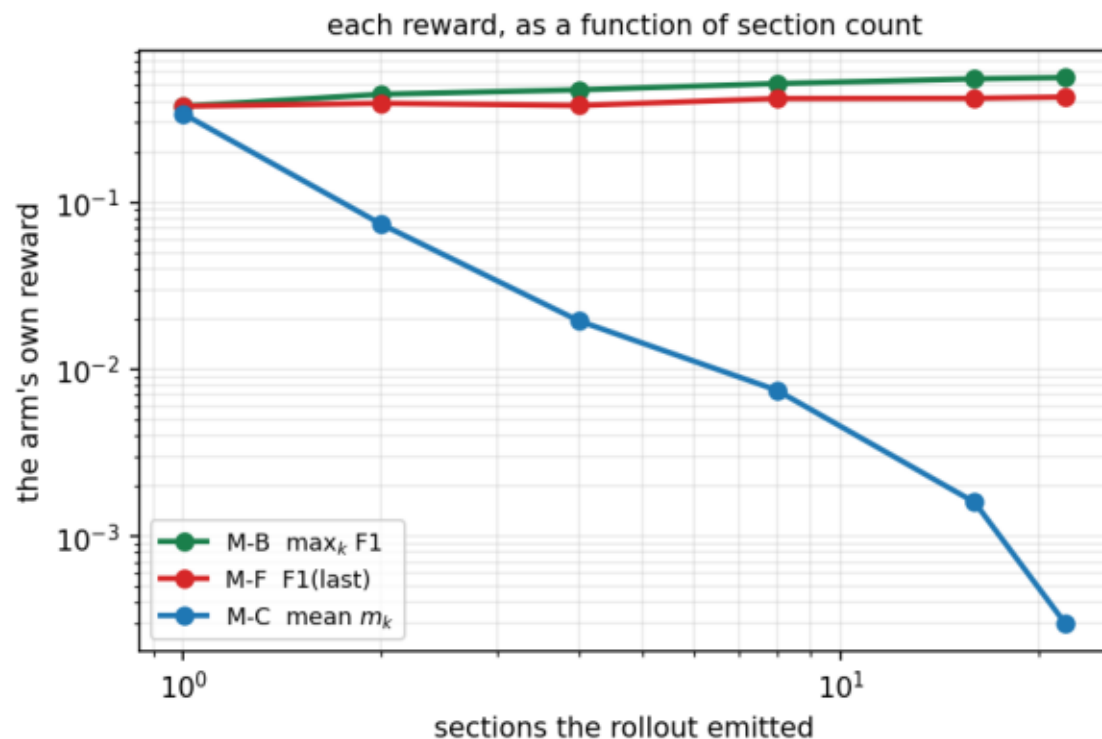


```
$ plot_reward_shape.py --data data --out plots
```

reward_vs_section_count.png

Each reward's gradient in the section-count direction predicts what that arm did: M-C falls (collapsed to 1.1), M-B rises (grew to ~26), M-F is nearly flat (unconstrained, ran to 259).

exp237 — every arm moved its section count in the direction its own reward pointed



section_count_incentive.png

m_k is not scale-free in the section count: at 1 section it is 366x its value at 22, while the rollout's own consensus falls 0.543 -> 0.341. Centring does not remove it -- the shorter rollout is the one above its group's mean.

exp237 — arm M-C's reward pays a rollout for emitting fewer sections

